

# Probability and simulation

POLI 210 · Seminar 2

Brad LeVeck  
September 9, 2026

# Today's questions

- What are the possible outcomes of a random process?
- Which outcomes count as the event we care about?
- When can we calculate a probability by counting?
- How can R simulate the same process?

**Reading:** PSD §§2.1, 2.2, and 2.4. Conditional probability and Bayes' rule are for next week.

# Why political scientists use probability

**Polling:** Even with a fixed population, different samples give different results.

**Experiments:** A random assignment procedure determines who receives treatment.

**Our task today:** Describe the procedure, then ask how often an event occurs.

The political examples use hypothetical numbers, not current polling estimates.

# The language of a probability model

| Term              | Meaning                             | Two coin tosses                |
|-------------------|-------------------------------------|--------------------------------|
| Random experiment | Procedure with an uncertain outcome | Toss twice                     |
| Outcome           | One complete result                 | $HT$                           |
| Sample space $S$  | Set of all possible outcomes        | $\{HH, HT, TH, TT\}$           |
| Event $E$         | A subset of $S$                     | Exactly one head: $\{HT, TH\}$ |

A **trial** is one repetition of the entire experiment. Random does **not** automatically mean equally likely.

# Describe outcomes before assigning probabilities

Two independent fair tosses give four equally likely ordered outcomes:

$$S = \{HH, HT, TH, TT\}.$$

But the possible **counts** of heads are not equally likely:

| Heads | Outcomes | Probability |
|-------|----------|-------------|
| 0     | $TT$     | $1/4$       |
| 1     | $HT, TH$ | $1/2$       |
| 2     | $HH$     | $1/4$       |

The representation of an outcome matters.

# Events are sets

| Notation        | Read it as                     | In R, for logical vectors E, F |
|-----------------|--------------------------------|--------------------------------|
| $E \cap F$      | Both $E$ and $F$               | $E \ \& \ F$                   |
| $E \cup F$      | $E$ or $F$ , including both    | $E \   \ F$                    |
| $\overline{E}$  | Not $E$                        | $!E$                           |
| $\emptyset$     | Impossible event               | No outcomes                    |
| $E \subseteq F$ | Every outcome in $E$ is in $F$ | $E$ implies $F$                |

**Disjoint events:**  $E \cap F = \emptyset$ . They cannot both occur in one trial.

# What makes a probability assignment valid?

1.  $P(E) \geq 0$  for every event  $E$ .
2.  $P(S) = 1$ .
3. Probabilities add for disjoint events:  $P(E \cup F) = P(E) + P(F)$  when  $E \cap F = \emptyset$ .

The third rule extends to any countable collection of pairwise disjoint events.

We can distribute probability unevenly across outcomes. The total must still be 1.

# Three useful consequences

## Complement

$$P(\overline{E}) = 1 - P(E).$$

## Union

$$P(E \cup F) = P(E) + P(F) - P(E \cap F).$$

**Containment:** If  $E \subseteq F$ , then  $P(E) \leq P(F)$ .

Why do we subtract the intersection in the union rule?



# A voting example: organize the overlap

In a hypothetical population, let:

- $E$ : plans to vote in the presidential election,  $P(E) = 0.55$ ;
- $F$ : plans to vote in the congressional election,  $P(F) = 0.36$ ;
- $P(E \cap F) = 0.35$ .

**With a neighbor:** Find the probability of planning to vote in at least one election, in neither, and in exactly one.

Start by splitting  $S$  into four disjoint pieces.

## Four disjoint pieces make the calculation clear

| Piece of the sample space | Probability                       |
|---------------------------|-----------------------------------|
| Both elections            | 0.35                              |
| Presidential only         | $0.55 - 0.35 = 0.20$              |
| Congressional only        | $0.36 - 0.35 = 0.01$              |
| Neither                   | $1 - (0.35 + 0.20 + 0.01) = 0.44$ |

At least one: 0.56. Exactly one: 0.21.

Check: the four disjoint pieces sum to 1.

# When counting gives a probability

If  $S$  is finite and its outcomes are **equally likely**,

$$P(E) = \frac{|E|}{|S|}.$$

Two independently rolled fair dice have  $6 \times 6 = 36$  equally likely ordered pairs.

Their sums run from 2 through 12, but those 11 sums are **not** equally likely.

**Checkpoint:** Would `sample(2:12, 1)` simulate the sum correctly?

# Enumerate a sample space in R

```
S <- expand.grid(first = 1:6, second = 1:6)  
head(S, 4)
```

|   | first | second |
|---|-------|--------|
| 1 | 1     | 1      |
| 2 | 2     | 1      |
| 3 | 3     | 1      |
| 4 | 4     | 1      |

```
nrow(S)
```

```
[1] 36
```

Each row is one complete outcome. This table is the full sample space, not a random sample from it.

# Select an event, then count it

```
E <- (S$first + S$second) == 7  
sum(E)           # number of favorable outcomes
```

```
[1] 6
```

```
mean(E)          # favorable / total
```

```
[1] 0.1666667
```

In R, TRUE acts like 1 and FALSE like 0 in these calculations.

This mean is an **exact probability** because the table contains every equally likely outcome once.

# sample() generates random draws

```
set.seed(210)  
sample(1:6, size = 12, replace = TRUE)
```

```
[1] 4 3 1 1 6 3 1 4 1 2 6 5
```

- `x`: the values to draw from (here 1:6).
- `size`: how many draws to return.
- `replace`: can the same entry be drawn again?
- `prob`: optional relative probabilities; the default is uniform.

`set.seed()` makes a simulation reproducible; it does not make a model correct.

# Replacement is a modeling choice

```
sample(1:6, size = 2, replace = TRUE)    # two die rolls  
sample(1:6, size = 2, replace = FALSE)   # two distinct slips
```

Dice may show the same number twice. Two distinct slips cannot be the same slip.

**Important:** One trial can contain multiple draws. To simulate two dice, one trial must return two rolls—not one roll and not all 36 possible pairs.

# A general simulation recipe

1. Describe one trial precisely.
2. Generate its complete outcome.
3. Record whether the event occurred: 1 or 0.
4. Repeat the trial many times.
5. Average those indicators.

$$\hat{p}_N = \frac{1}{N} \sum_{i=1}^N I_i, \quad I_i = \begin{cases} 1 & \text{event occurs in trial } i, \\ 0 & \text{otherwise.} \end{cases}$$

The result estimates the probability **under the model we simulated**.



# Simulate the two-dice event

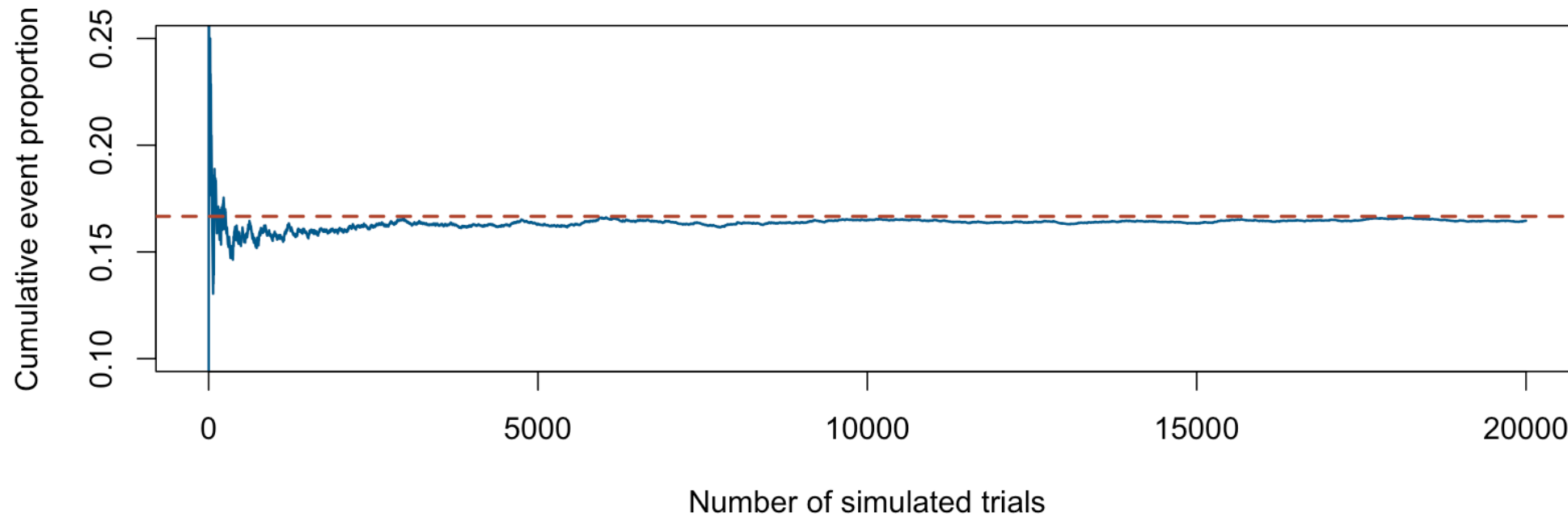
```
set.seed(211)
N <- 20000
first <- sample(1:6, N, replace = TRUE)
second <- sample(1:6, N, replace = TRUE)
hit <- (first + second) == 7
c(estimate = mean(hit), exact = 6 / 36)
```

```
estimate    exact
0.1646000 0.1666667
```

Entry  $i$  in each vector belongs to trial  $i$ .

The Monte Carlo estimate usually differs slightly from the exact probability.

# More trials stabilize the estimate



The dashed line is the exact probability,  $1/6$ . The law of large numbers supports stabilization for independent repeated trials from the same model. Accuracy need not improve at every step.

# A randomized experiment: define one assignment

Three participants are each assigned treatment by an independent fair coin.

```
set.seed(212)
assignment <- sample(c("C", "T"), 3, replace = TRUE)
assignment
```

```
[1] "T" "T" "C"
```

```
any(assignment == "T") & any(assignment == "C")
```

```
[1] TRUE
```

Our event: **both treatment and control are represented.**

This is a question about the assignment procedure, before observing any outcomes.

# The experiment's sample space

$$S = \{CCC, CCT, CTC, CTT, TCC, TCT, TTC, TTT\}.$$

With independent fair assignments, each pattern has probability  $1/8$ .

Six patterns include both groups:

$$P(\text{both groups represented}) = \frac{6}{8} = \frac{3}{4}.$$

**Design question:** Does this procedure guarantee a usable comparison group?

# replicate() repeats the whole trial

```
set.seed(213)
mixed <- replicate(20000, {
  a <- sample(c("C", "T"), 3, replace = TRUE)
  any(a == "T") & any(a == "C")
})
mean(mixed)
```

```
[1] 0.74855
```

The last expression inside { ... } is the result saved from each trial.

Put `set.seed()` **before** `replicate()`, not inside it.

# A different design has a different sample space

Suppose we require exactly one treated participant:

```
sample(c("T", "C", "C"), size = 3, replace = FALSE)
```

```
[1] "C" "C" "T"
```

Now  $S = \{TCC, CTC, CCT\}$ . Each pattern has probability  $1/3$ .

Both groups are represented with probability 1.

We changed the assignment procedure—not just the R syntax.

# A hypothetical polling model

Assume each sampled response is an independent approval indicator with  $P(\text{approve}) = 0.60$ .

```
set.seed(214)
responses <- sample(c(0, 1), 20, replace = TRUE,
                    prob = c(0.40, 0.60))
mean(responses)
```

```
[1] 0.5
```

1 means approve. The sample mean is the approval share.

For a large population, this is a simplified model of repeated sampling.

# Weighted outcomes: calculate and simulate

For two independent responses with approval probability 0.60, the ordered outcomes are **not** equally likely:

| Outcome              | Probability under independent sampling |
|----------------------|----------------------------------------|
| Neither approves: 00 | $0.40 \times 0.40 = 0.16$              |
| First only: 10       | $0.60 \times 0.40 = 0.24$              |
| Second only: 01      | $0.40 \times 0.60 = 0.24$              |
| Both approve: 11     | $0.60 \times 0.60 = 0.36$              |

Agreement is  $\{00, 11\}$ , so  $P(\text{agree}) = 0.16 + 0.36 = 0.52$ . Multiply within an independent pair; add across disjoint matching outcomes.



# Repeat the poll, not just one response

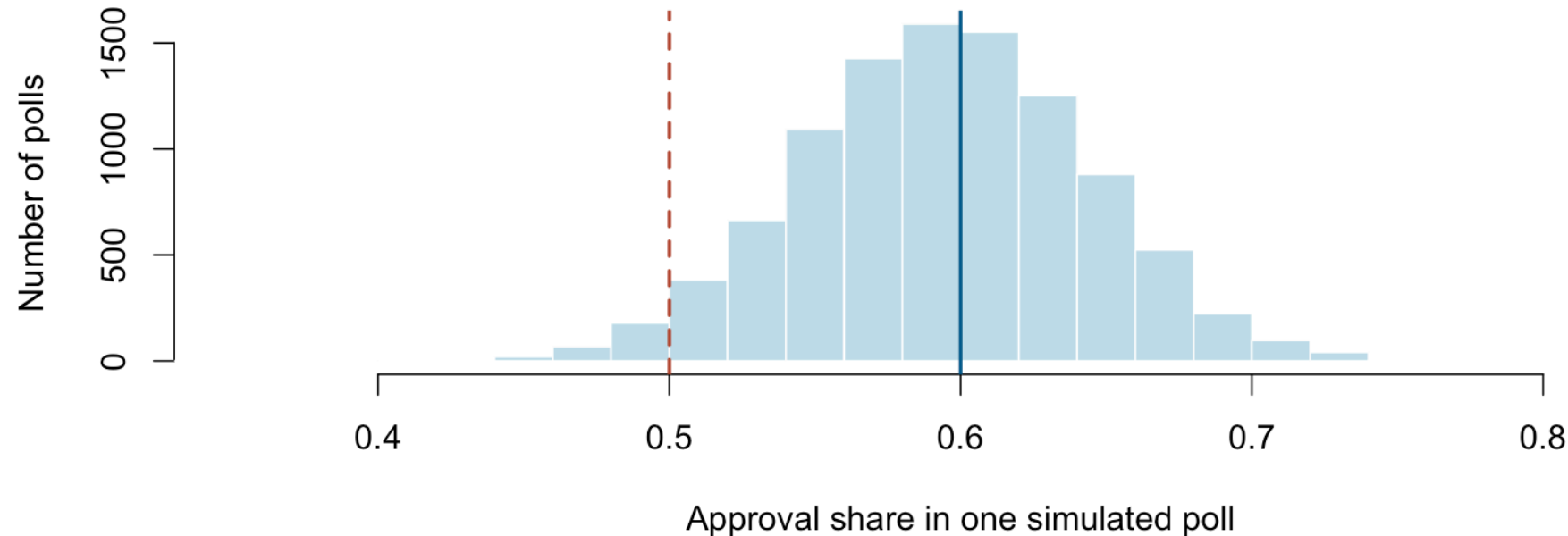
What is the chance that a poll of 100 reports **less than 50% approval**?

```
set.seed(215)
poll_share <- replicate(10000, {
  x <- sample(c(0, 1), 100, replace = TRUE,
             prob = c(0.40, 0.60))
  mean(x)
})
mean(poll_share < 0.50)
```

```
[1] 0.0168
```

There are 100 responses per trial and 10,000 trials.

# Different polls, one assumed population



Dashed line: 0.50; solid line: the assumed 0.60 approval rate. This captures sampling variability under our assumptions. It does not fix nonresponse, question wording, or an unrepresentative sampling process.

# Counting without listing every outcome

**Product rule:** If one choice has  $m$  options and the next has  $n$  options for each first choice, there are  $mn$  ordered possibilities.

- Two dice:  $6 \times 6 = 36$  ordered outcomes.
- Twenty yes/no responses:  $2^{20} = 1,048,576$  response sequences.
- Three distinct names chosen in order from six:  $6 \times 5 \times 4 = 120$ .

Counting outcomes does not, by itself, tell us they are equally likely. The polling sequences are not equally likely when approval probability is 0.60.

# Order and replacement determine the count

| Procedure                                      | Number of possibilities          |
|------------------------------------------------|----------------------------------|
| Ordered, $k$ draws from $n$ , with replacement | $n^k$                            |
| Ordered, $k$ distinct choices from $n$         | $n!/(n - k)!$                    |
| Unordered, $k$ distinct choices from $n$       | $\binom{n}{k} = n!/[k!(n - k)!]$ |

For choices without replacement,  $0 \leq k \leq n$ ; also  $0! = 1$ .

Use `factorial(n)` and `choose(n, k)` in R.

**Check:** Are a committee and a ranked list the same kind of outcome?

## A committee example

Choose a two-person committee uniformly from six legislators: four incumbents and two newcomers.

$$P(\text{both incumbents}) = \frac{\binom{4}{2}}{\binom{6}{2}} = \frac{6}{15} = 0.40.$$

```
choose(4, 2) / choose(6, 2)
```

```
[1] 0.4
```

The committee is unordered. Every pair of distinct legislators is equally likely.

# Simulate the same committee procedure

```
set.seed(216)
both_incumbents <- replicate(20000, {
  committee <- sample(1:6, 2, replace = FALSE)
  all(committee <= 4) # legislators 1-4 are incumbents
})
mean(both_incumbents)
```

```
[1] 0.3978
```

**Your turn:** Find the probability of exactly one newcomer by counting. Then change the final expression to simulate that event.

# Before trusting a simulation

- Does one trial represent the intended process?
- Did we choose the right replacement rule and weights?
- Is the event coded correctly, including equality and boundary cases?
- Are all repetitions generated afresh?
- Is the answer plausible, and does it agree with an exact calculation when available?

Running an incorrect model more times produces a more stable wrong answer.

## A2: calculation and simulation

**Due:** Wednesday, September 16, at 1:30 p.m.

PSD exercises **2.1, 2.3, 2.4, 2.7, 2.9, 2.33, and 2.36.**

- Read each linked exercise; write explanations in the Quarto report.
- Save code in `R/a2_answers.R`; run the visible checks locally.
- Render, commit, push, and check GitHub Actions.

Next week: conditional probability, independence, and Bayes' rule; Blitzstein and Hwang, Chapter 2.



## Sources and scope

Speegle and Clair, *Probability, Statistics, and Data*, Chapter 2, especially §§2.1, 2.2, and 2.4.

The voting example adapts prior POLI 210 probability materials. Polling and three-person random-assignment examples build on themes in Matthew Blackwell's *Gov 2000: Random Variables* (Fall 2016), retained in the earlier course archive.

The simulations and numerical polling assumptions here are instructional examples. Formal random-variable distributions and inference come later in the course.